

Project 5: Intelligent Patient Support Chatbot with Hallucination and PII Guardrails

This You will build a simple patient support chatbot that answers user queries based on a provided knowledge base.

You will then integrate **Comet Opik** and either **Langfuse** or **Langsmith** to observe its behavior and implement **guardrails** to **prevent common LLM pitfalls: hallucination and Personally Identifiable Information (PII) leakage**.

Core concepts to be demonstrated

- **Observability:**
 - **Tracing:** Tracking the flow of requests through the LLM application (prompt, model call, response generation, guardrail checks).
 - **Logging:** Capturing input prompts, LLM outputs, intermediate steps, and guardrail decisions.
 - **Metrics:** Monitoring token usage, latency, success/failure rates of LLM calls and guardrail activations.
 - **Evaluation:** Automatically assessing the quality of LLM responses (e.g., factual accuracy, relevance) using LLM-as-a-Judge or rule-based evaluations.
 - **Debugging:** Identifying why the LLM produced a particular output, especially when it's incorrect or undesirable.
- **Guardrails:**
 - **Hallucination Detection:** Identifying when the LLM generates information not present in the provided knowledge base.
 - **PII Redaction/Blocking:** Preventing the LLM from outputting sensitive personal information (e.g., names, addresses, phone numbers, medical record numbers).

Tools to be Used:

- **LLM Framework:** LangChain (for building the LLM application structure and facilitating integrations).
- **LLM Provider:** Amazon Bedrock (any model)

- **Observability/Evaluation Platform 1:** Comet Opik (for LLM evaluation, tracing, and monitoring).
- **Observability/Evaluation Platform 2:** Langfuse OR Langsmith (choose one based on preference/familiarity, both offer similar core functionalities for tracing and evaluation).

Compelling Use Case: Patient Support Chatbot for a Fictional Hospital

Scenario: A large hospital, "CareBliss," wants to improve patient communication and reduce the burden on its staff. They've implemented an LLM-powered chatbot to answer common patient questions.

Challenges this demo addresses:

1. **Hallucinations:** The chatbot might invent medical advice or incorrect information about hospital procedures, which is extremely dangerous.
2. **PII Leakage:** Patients might share sensitive information (e.g., "My medical record number is 98765...") which the chatbot must not expose.
3. **Lack of Transparency:** Understanding why the chatbot gave a particular answer is crucial for safety and trust in a medical setting.

Mini-Project Structure & Implementation Details:

1. Setup

Knowledgebase:

Create a simple knowledgebase text file like below:

Service: Cardiology

Description: We provide comprehensive cardiac care, including diagnosis and treatment of heart conditions.

Hours: Monday-Friday, 8:00 AM - 5:00 PM

Contact: 98982132667

Service: Radiology

Description: We offer a full range of imaging services, including X-rays, CT scans, and MRIs.

Hours: Monday-Saturday, 7:00 AM - 7:00 PM

Contact: 9899445667

Appointment Scheduling: You can schedule appointments online at carebliss.com/appointments or by calling our main line at +91-22-111-2222.

2. Build the app and integrate Observability framework(s) into it

Environment Variables:

COMET_API_KEY, COMET_WORKSPACE, COMET_PROJECT_NAME

LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY, LANGFUSE_HOST (if using Langfuse)

LANGCHAIN_TRACING_V2=true, LANGCHAIN_API_KEY, LANGCHAIN_PROJECT (if using Langsmith)

Install Libraries

```
pip install langchain openai pydantic comet_llm langfuse # or langsmith
```

Develop your app

```
import os
```

```
from langchain.chains import RetrievalQA
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain.vectorstores import FAISS
```

```
from langchain.embeddings import OpenAIEmbeddings
```

```
from langchain.text_splitter import CharacterTextSplitter
```

```
from langchain.document_loaders import TextLoader
```

```
# Initialize Langfuse/Langsmith (choose one)
```

```
# If using Langfuse
```

```
# from langfuse.callback import CallbackHandler
```

```
# langfuse_handler = CallbackHandler()
```

```
# If using Langsmith (set environment variables as needed)
```

```
# from langsmith import traceable # Though Langchain tracing env var often sufficient
```

```
# Initialize Comet Opik
```

```
import comet_llm
```

```
comet_llm.init() # Ensure COMET_API_KEY, COMET_WORKSPACE,  
COMET_PROJECT_NAME are set
```

```
# Load knowledge base
```

```

loader = TextLoader("knowledge_base.txt")
documents = loader.load()
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
texts = text_splitter.split_documents(documents)
embeddings = OpenAIEmbeddings()
docsearch = FAISS.from_documents(texts, embeddings)

# Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0) # Consider higher temp for
more "creative" (hallucinatory) responses initially

# Create a retrieval-augmented generation (RAG) chain
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=docsearch.as_retriever(),
    return_source_documents=True
)

def chat_with_bot(query: str):
    print(f"\nUser Query: {query}")
    # Integrate with Langfuse/Langsmith (pass callback handler if needed)
    # config = {"callbacks": [langfuse_handler]} if 'langfuse_handler' in globals() else {}
    # result = qa_chain.invoke({"query": query}, config=config)

    # For simplicity, we'll rely on environment variables for Langsmith/Comet for this
    basic demo.

    # More explicit integration would use callback handlers.
    result = qa_chain.invoke({"query": query})

    response = result["result"]

```

```

source_documents = result["source_documents"]

print(f"Chatbot Response: {response}")

print(f"Source Documents: {[doc.metadata for doc in source_documents]}")

return response, source_documents

```

3. Guardrails implementation

Implement simple, rule-based guardrails for demonstration. In a real-world scenario, you'd use much more sophisticated techniques.

```

import re

from typing import Tuple, List

# PII Detection (simplified regex-based)
def detect_pii(text: str) -> List[str]:
    pii_found = []

    # Simple regex for phone numbers, email, and medical record numbers (for demo)
    if re.search(r'\b\d{10}\b', text): # 10 digits
        pii_found.append("Phone Number")

    if re.search(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', text):
        pii_found.append("Email Address")

    if re.search(r'\bMRN\s*:\s*\d+\b', text, re.IGNORECASE): # Medical Record Number
        pii_found.append("Medical Record Number")

    return pii_found

# Hallucination Detection (simplified - checks for keywords NOT in knowledge base)
# In a real scenario, this would involve comparing LLM output to retrieved chunks
# or using an LLM-as-a-judge for factual consistency.
def detect_hallucination(response: str, source_documents: List[str]) -> bool:
    knowledge_base_text = " ".join([doc.page_content for doc in source_documents])

    # Very basic: check if a key phrase from response is *not* in source documents
    # This is a highly simplified hallucination check for demo purposes.

```

```
# Real-world scenarios require much more robust methods (e.g., factual consistency evaluation).
```

```
response_lower = response.lower()
```

```
# Example hallucination check (simplified):
```

```
# If the response talks about "alternative treatments" but it's not in the knowledge base.
```

```
if "alternative treatments" in response_lower and "alternative treatments" not in knowledge_base_text.lower():
```

```
    return True
```

```
return False
```

```
def apply_guardrails(response: str, source_documents: List[str]) -> Tuple[str, List[str], List[str]]:
```

```
    pii_issues = detect_pii(response)
```

```
    hallucination_detected = detect_hallucination(response, source_documents)
```

```
    guardrail_alerts = []
```

```
    final_response = response
```

```
    if pii_issues:
```

```
        guardrail_alerts.append(f"PII Detected: {' '.join(pii_issues)}. Response redacted.")
```

```
        # Simple redaction for demo
```

```
        final_response = re.sub(r'\b\d{10}\b', '[REDACTED_PHONE]', final_response)
```

```
        final_response = re.sub(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', '[REDACTED_EMAIL]', final_response)
```

```
        final_response = re.sub(r'\bMRN\s*:\s*\d+\b', '[REDACTED_MRN]', final_response, re.IGNORECASE)
```

```
    if hallucination_detected:
```

```
        guardrail_alerts.append("Potential Hallucination Detected. Response might contain unverified information.")
```

```
# In a real scenario, you might regenerate the response or flag for human review.
```

```
# For this demo, we'll just add an alert.
```

```
return final_response, pii_issues, guardrail_alerts
```

4. Integrating with Comet Opik and Langfuse/Langsmith

Both Comet Opik and Langfuse/Langsmith offer LangChain callbacks, making integration straightforward.

For LangChain, Comet Opik, and Langfuse/Langsmith:

The langchain library automatically integrates with these platforms if the respective environment variables are set and the callbacks are enabled (e.g., `LANGCHAIN_TRACING_V2=true` for Langsmith, and `comet_llm.init()` for Comet Opik).

You can then run your `qa_chain.invoke` and the tracing data will be sent to the configured platforms.

5. Main demo loop:

```
def run_demo():
```

```
    print("Welcome to the HealthHaven Patient Support Chatbot!")
```

```
    print("Type 'exit' to end the conversation.")
```

```
    while True:
```

```
        user_input = input("\nYour query: ")
```

```
        if user_input.lower() == 'exit':
```

```
            print("Thank you for using HealthHaven Support!")
```

```
            break
```

```
        original_response, source_docs = chat_with_bot(user_input)
```

```
        final_response, pii_issues, guardrail_alerts = apply_guardrails(original_response,
[doc.page_content for doc in source_docs])
```

```
        print(f"\n--- Post-Guardrail Analysis ---")
```

```
        if guardrail_alerts:
```

```

    for alert in guardrail_alerts:

        print(f" 🚨 {alert}")

    print(f"Final (Guarded) Chatbot Response: {final_response}")

else:

    print("✅ No guardrail issues detected.")

    print(f"Final Chatbot Response: {final_response}")


# Log to Comet Opik for specific evaluation of guardrail outcomes
comet_llm.log_prompt(
    prompt=user_input,
    output=final_response,
    metadata={
        "original_llm_output": original_response,
        "source_documents": [doc.page_content for doc in source_docs],
        "pii_detected": pii_issues,
        "hallucination_flag": "True" if "Potential Hallucination Detected" in
guardrail_alerts else "False",
        "guardrail_alerts": guardrail_alerts,
        "model": llm.model_name
    }
)

print("Logged interaction to Comet Opik and Langfuse/Langsmith (via LangChain
callbacks).")


if __name__ == "__main__":
    run_demo()

```

Suggested Presentation/Demo Flow

o Title: "Seeing and Securing Your LLM Chatbot: A HealthHaven Case Study"

Objective: To demonstrate how Comet Opik and Langfuse/Langsmith provide critical observability and guardrail capabilities for an LLM-powered patient support chatbot.

Pre-requisites:

- Python environment with necessary libraries (langchain, openai, pydantic, comet_llm, langfuse or langsmith).
- API keys for OpenAI, Comet, and Langfuse/Langsmith configured as environment variables.
- knowledge_base.txt file in the same directory.

Demo Steps:

(1. Introduction & Setup)

- "Welcome to the demo! We're simulating a patient support chatbot for a hospital called CareBliss."
- "LLM applications, while powerful, can be unpredictable. We need ways to **observe** what they're doing internally and implement **guardrails** to prevent undesirable behavior, especially in a sensitive domain like healthcare."
- "Today, we'll see how **Comet Opik** and **Langfuse/Langsmith** help us achieve this, working seamlessly with our LangChain chatbot."
- "Show the knowledge_base.txt file. Explain that the chatbot should only answer based on this."
- "Briefly show the Python code, highlighting comet_llm.init() and the LangChain RetrievalQA setup. Mention how LangChain callbacks handle the tracing integration automatically."

(2. Normal Operation & Observability)

- "Let's start with a typical, well-behaved query."
- **Run the script.**
- **User Input 1:** "What are the hours for the Cardiology service?"
 - *Expected Output:* Correct response from knowledge_base.txt.
 - *Demo Narration:* "Notice the chatbot provides a concise, accurate answer. Now, let's look behind the scenes."
 - "Go to **Langfuse/Langsmith UI**: Show the trace for this interaction. Point out the prompt, the LLM call, the retrieved source documents, and the final response. Emphasize the latency and token usage metrics."
 - "Go to **Comet Opik UI**: Show the logged prompt and output. Highlight the metadata section, where we custom-logged the source documents, and

initial guardrail flags (which should be 'False'). Explain that Comet Opik also tracks latency and token usage."

(3. Demonstrating Hallucination (and its detection))

- "Now, let's try to make the chatbot 'hallucinate' – generate information not in its knowledge base. This is especially critical to avoid in healthcare."
- **User Input 2:** "Does the hospital offer alternative treatments like acupuncture?"
 - *Expected Output:* The chatbot might try to answer, but our simple guardrail should flag it. The response will *likely* include the guardrail alert.
 - *Demo Narration:* "The chatbot attempted to answer, but our (simple) hallucination guardrail detected that 'alternative treatments' or 'acupuncture' wasn't in its knowledge base. In a real system, this should trigger a 'I don't have information on that' response, or flag for immediate human review."
 - "Go back to **Langfuse/Langsmith UI**: Show the trace. You might see the LLM's raw output before the guardrail acted. Observe the source documents retrieved (they won't contain "acupuncture")."
 - "Go back to **Comet Opik UI**: Show the new log. Crucially, point out that `hallucination_flag` in the metadata is now `True`, and the `guardrail_alerts` list contains the hallucination warning. Explain how this allows us to quickly identify and analyze such instances."

(4. Demonstrating PII Leakage Prevention)

- "Next, we'll test our PII guardrail. A patient might inadvertently share sensitive info."
- **User Input 3:** "I need to schedule an appointment. My email is patient@email.com and my medical record number is MRN: 123456."
 - *Expected Output:* The chatbot should respond, but the email and MRN in its *output* should be redacted.
 - *Demo Narration:* "See how the chatbot's response has redacted the email and medical record number? Our PII guardrail caught it before it could be exposed."
 - "Go to **Langfuse/Langsmith UI**: Observe the trace. You might see the original LLM output containing the PII, and then the final, redacted output. This demonstrates the "before and after" effect of guardrails."

- "Go to **Comet Opik UI**: Show the log. Highlight pii_detected now listing 'Email Address', 'Medical Record Number', and the guardrail_alerts showing the redaction message. This gives us clear evidence of the guardrail's effectiveness."

(5. Combined Scenario & Debugging Capabilities)

- "Let's combine a few challenges."
- **User Input 4**: "Tell me how to schedule an appointment and my email is test@email.com. Also, what are the hours for the neurology department?"
 - *Expected Output*: Appointment scheduling info (from KB), email redacted, and a response indicating it doesn't have info on neurology (since it's not in the KB).
 - *Demo Narration*: "Here, we see both guardrails in action. The PII is redacted, and the chatbot correctly addresses the appointment scheduling while avoiding hallucination about a non-existent department."
 - "Use **Langfuse/Langsmith** to drill down into the trace. If the LLM struggled with the mixed query, you could see intermediate thoughts, tool calls (if more complex), and the steps leading to the final (guarded) response. This is invaluable for debugging, especially in complex medical scenarios."
 - "Use **Comet Opik** to compare this run with previous ones. Show how you can filter by pii_detected or hallucination_flag to quickly find problematic interactions and analyze patterns over time."

(6. Conclusion)

- "As you've seen, **Comet Opik** and **Langfuse/Langsmith** provide powerful capabilities for LLM observability and guardrails, which are *especially* critical in healthcare."
- "They allow us to:"
 - "**Understand** exactly what our LLM applications are doing (tracing, logging)."
 - "**Measure** their performance (metrics, evaluations)."
 - "**Catch** and **prevent** dangerous behaviors like hallucinations and PII leakage."

- "**Debug** issues efficiently and **iterate** on our LLM applications with confidence, knowing we have robust safety measures in place."
- "This combination of tools is essential for building responsible, reliable, and production-ready LLM applications, particularly in sensitive fields like medicine where patient safety and data privacy are paramount."